

# UNIVERSIDAD DE GUADALAJARA



## CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

### Algoritmo Lowest Common Ancestor con Binary Lifting Divide y Vencerás

**Materia:** Análisis De Algoritmos

**Sección:** D01

**Profesor:** Jorge Ernesto López Arce Delgado

**Equipo:**

- **Nombre:** IA
- **Integrantes:**
  - Ismael Gándara Cornejo
  - Angel Miguel Villalvazo Vazquez

**Carrera:** Ingeniería en Computación

**Fecha:** 26 de octubre del 2025

# Índice

|                                    |   |
|------------------------------------|---|
| • Introducción .....               | 3 |
| • Objetivo .....                   | 3 |
| • Roles y responsabilidades .....  | 3 |
| • Desarrollo .....                 | 4 |
| • Conclusión .....                 | 7 |
| • Referencias Bibliográficas ..... | 8 |

## Índice de imágenes

|                                                                          |   |
|--------------------------------------------------------------------------|---|
| • Fig. 1. Interfaz principal del programa. ....                          | 4 |
| • Fig. 2. Simulando la DFS para medir profundidades y padres directos... | 5 |
| • Fig. 3. Construcción de la tabla de saltos logarítmicos. ....          | 5 |
| • Fig. 4 Gráfica de complejidades a través de los números de búsquedas.  | 6 |
| • Fig. 5. Gráfica ideal y/o esperada de las complejidades. ....          | 6 |

## Introducción

Encontrar el Lowest Common Ancestor o LCA es un problema muy conocido y utilizado en concursos de algoritmia. Este problema se da dado un árbol y dos nodos  $u$  y  $v$ , donde el LCA se puede definir como el nodo más alejado de la raíz que es ancestro tanto de  $u$  como de  $v$ . Esto tiene diferentes aplicaciones en áreas como el manejar sistemas de archivos, redes (enrutamiento) y bioinformática (análisis de árboles filogenéticos). Nuestro código también agregamos un visualizador para comparar dos métodos del mismo algoritmo. Uno es el LCA de Fuerza Bruta que consiste en subir desde los nodos  $u$  y  $v$  hacia la raíz, un padre a la vez, lo cual es ineficiente para árboles grandes o un gran número de consultas. En cambio, el LCA de Divide y Vencerás (Binary Lifting) es un método mucho más avanzado y eficiente que invierte tiempo en una fase de preprocesamiento y después, para encontrar el LCA, los nodos dan saltos  $2^i$  hasta donde se necesite.

## Objetivo

El propósito de este proyecto es implementar el algoritmo de divide y vencerás, agregando la programación dinámica y a su vez medimos su complejidad en tiempo y espacio, demostrando por qué la inversión en preprocesamiento del Binary Lifting ofrece una gran ventaja. Además, explicaremos el uso de este algoritmo en distintas áreas como las anteriores mencionadas.

El objetivo general de esta actividad crear un programa usando Python y la librería Tkinter, la cual nos servirá como herramienta para poder observar, y comparar la eficiencia de los dos algoritmos para encontrar el LCA. Implementaremos las lógicas de ambos algoritmos, incluyendo también las fases necesarias para la consulta del LCA con Binary Lifting. Por último, crearemos una GUI que permite al usuario seleccionar entre qué nodos y algoritmos se realizará la búsqueda del LCA, integrando herramientas de medición como `time.perf_counter` y `tracemalloc`, y graficando los resultados con `matplotlib`.

## Roles Y Responsabilidades

| Responsable                                               | Tareas                                  | Fecha      | Entregable                                 |
|-----------------------------------------------------------|-----------------------------------------|------------|--------------------------------------------|
| Angel Miguel Villalvazo Vazquez                           | Investigación teórica                   | 18/10/2025 | Conocimiento para el proyecto              |
| Angel Miguel Villalvazo Vazquez                           | Tests                                   | 20/10/2025 | Reporte de pruebas                         |
| Angel Miguel Villalvazo Vazquez<br>Ismael Gándara Cornejo | Código en Python                        | 21/10/2025 | Archivo .py                                |
| Ismael Gándara Cornejo                                    | Código de visualizaciones y complejidad | 21/10/2025 | Nuevo archivo .py con las implementaciones |
| Ismael Gándara Cornejo                                    | Presentación                            | 26/10/2025 | Presentación en Canva                      |

|                                                           |                        |            |                  |
|-----------------------------------------------------------|------------------------|------------|------------------|
| Angel Miguel Villalvazo Vazquez<br>Ismael Gándara Cornejo | Reporte De<br>Proyecto | 26/10/2025 | Documento en pdf |
|-----------------------------------------------------------|------------------------|------------|------------------|

## Desarrollo

Usando Python, la base que usamos fue la biblioteca tkinter, con la cual construimos toda la GUI y el componente Canvas, donde se dibuja el estado del árbol de forma interactiva. La estructura de datos del árbol se inicializa desde una lista de aristas (self.aristas) y se transforma en una lista de adyacencia (self.lista\_adyacencia) para facilitar los recorridos.

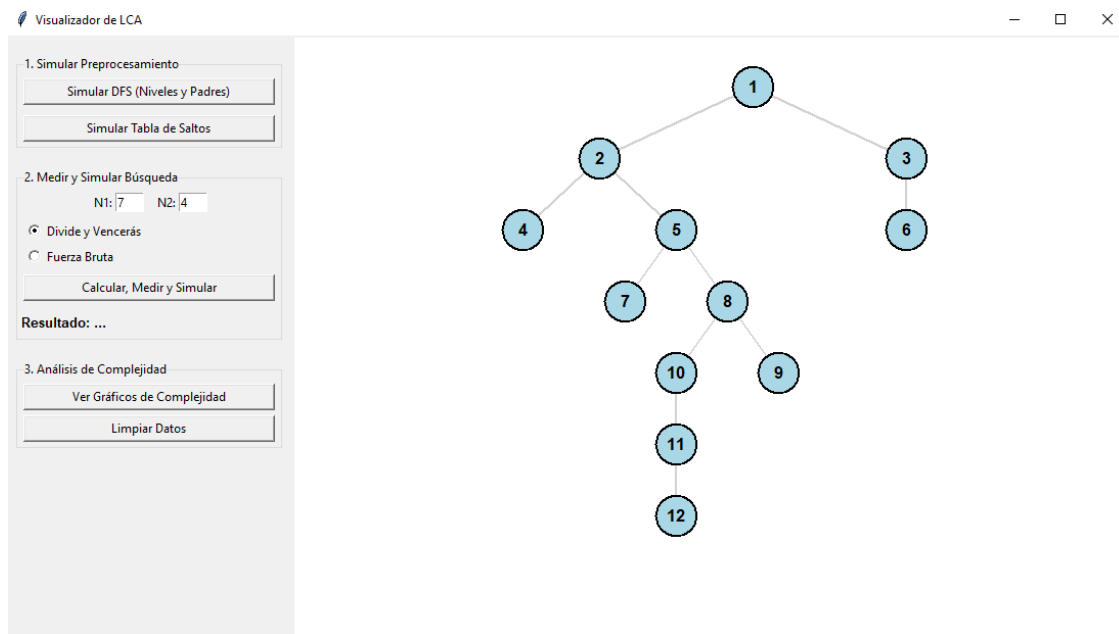


Fig. 1. Interfaz principal del programa.

La parte del desarrollo del preprocesamiento del algoritmo de Binary Lifting, encontrada en la función `preprocesar_algoritmos`, inicia con un DFS para calcular el nivel de cada nodo (`self.nivel`) y su padre directo (`self.padre[u][0]`).

Visitando nodo: 9  
Padre: 8  
Nivel: 5

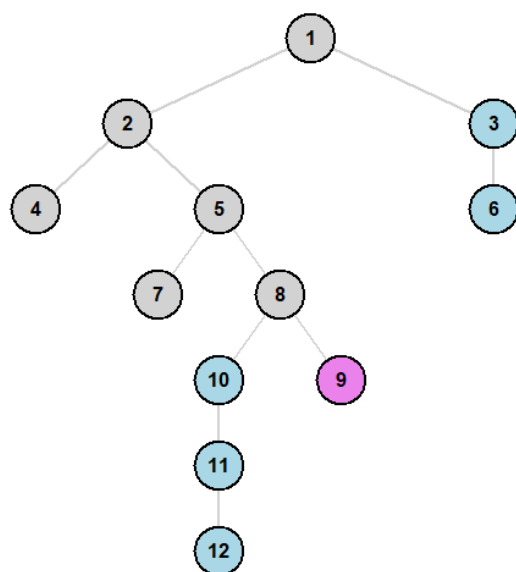


Fig. 2. Simulando la DFS para medir profundidades y padres directos.

Después, se construye la tabla de saltos mediante aplicando ahí la programación dinámica, y después, se implementan las dos lógicas de consulta (fuerza bruta y binary lifting).

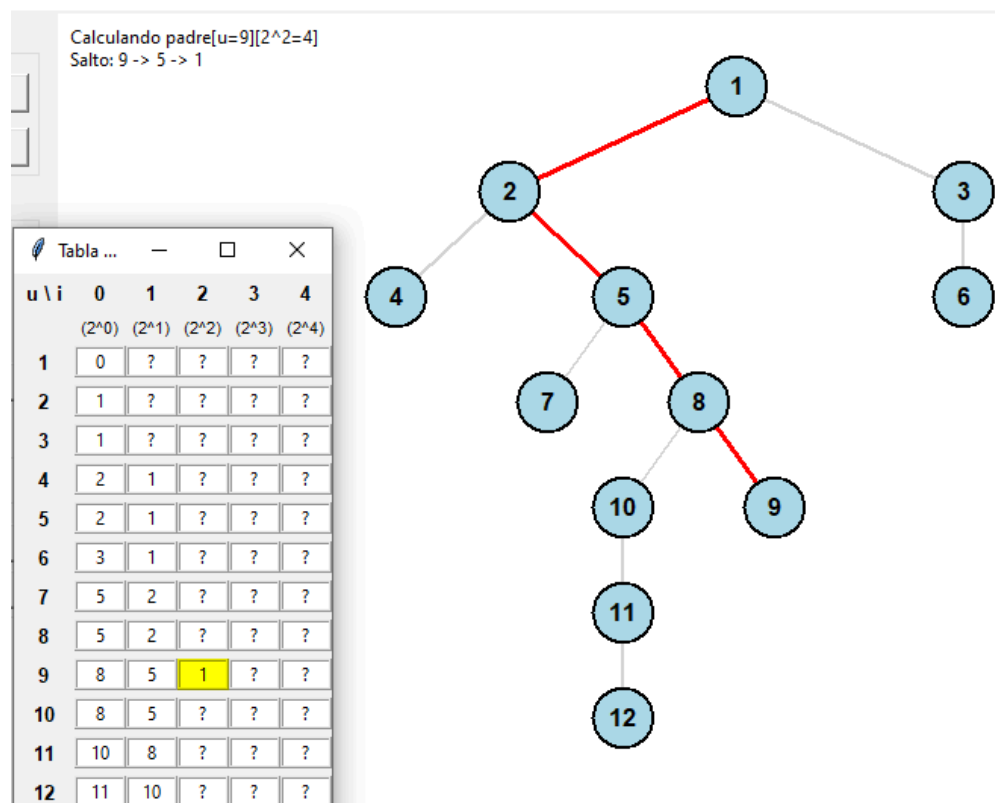


Fig. 3. Construcción de la tabla de saltos logarítmicos.

La función `obtener_lca_fuerza_bruta` utiliza bucles `while` para primero nivelar los nodos dependiendo de la profundidad de cada uno y una vez estén nivelados subirlos juntos hasta que se unen. Por otro lado, la función `obtener_lca` implementa el método de Binary Lifting utilizando la tabla de saltos precalculada para nivelar los nodos y encontrar el ancestro común, dando "saltos" con base 2, iterando desde el salto más grande hacia abajo.

Para lograr la simulación, tenemos varias funciones como `dfs_generador` y `obtener_lca_generador` usadas como generadores de Python, usando `yield` para pausar su estado después de cada paso. La función `ejecutar_medicion_y_simulacion` encierra la ejecución y usa `time.perf_counter` para mediciones de tiempo y `tracemalloc` para rastrear el uso de memoria. Estos resultados se van agregando en la lista `self.datos_rendimiento` para que, con la función `graficar_complejidad_medida`, `matplotlib` lea la lista y se dibujen los gráficos de complejidad de los algoritmos, demostrando la diferencia de eficiencia entre los dos.

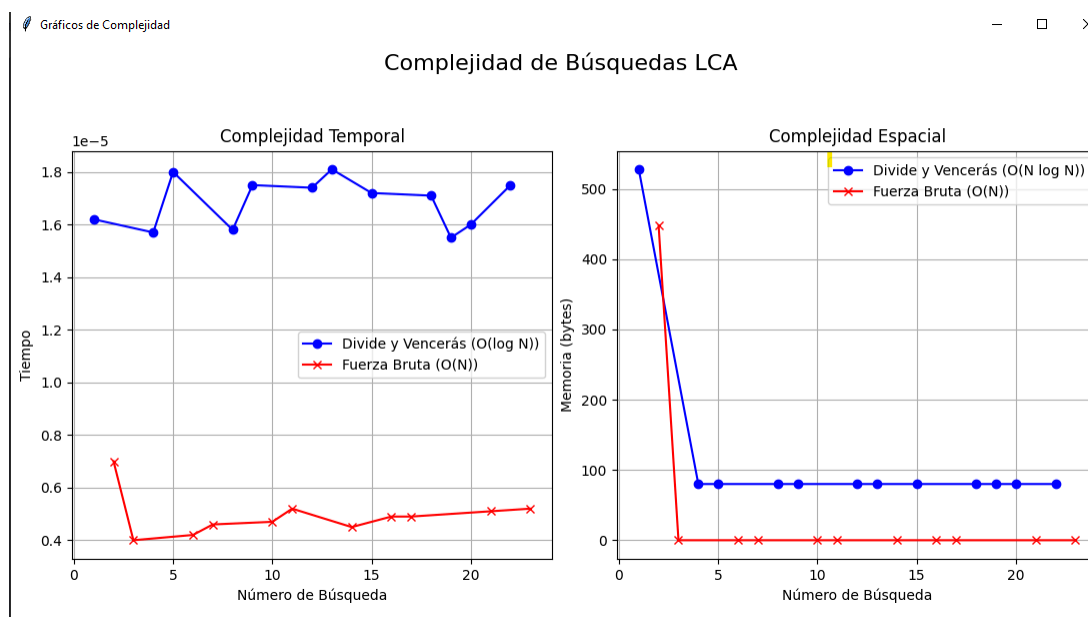


Fig. 4. Gráfica de complejidades a través de los números de búsquedas.

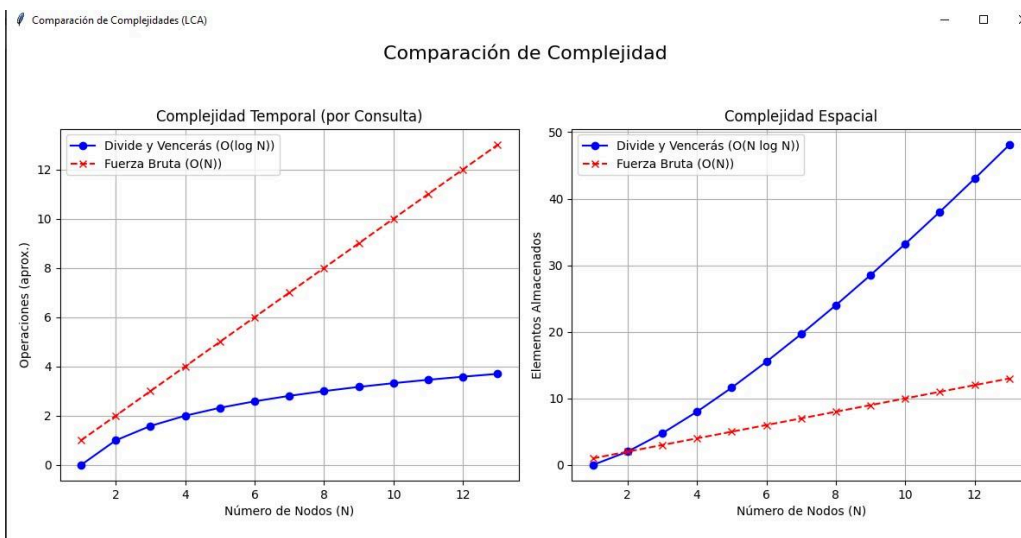


Fig. 5. Gráfica ideal y/o esperada de las complejidades.

## Conclusión

Con esta práctica demostramos la diferencia de eficiencia entre los métodos que puede tener un algoritmo en tiempo real. Mientras la Fuerza Bruta subía visualmente un nodo a la vez, el Binary Lifting realizaba saltos logarítmicos, diferencia que después fue comprendida con las gráficas de complejidad generadas, las cuales mostraron picos de tiempo más altos en el caso del algoritmo con Fuerza Bruta, en contraste con el rendimiento del Binary Lifting (ocurre de manera viceversa cuando hablamos de complejidad espacial). Además, la actividad nos demostró el poder del preprocesamiento a la hora de hacer las búsquedas, ya que aunque el Binary Lifting requiere una complejidad inicial de  $O(N \log N)$ , el ahorro en cada consulta lo complementa bastante bien, siendo una buena opción para estructuras más grandes.

## Referencias Bibliográficas

- Lowest common ancestor - binary lifting - algorithms for competitive programming. (s/f). Cp-algorithms.com. Recuperado el 27 de octubre de 2025, de [https://cp-algorithms.com/graph/lca\\_binary\\_lifting.html](https://cp-algorithms.com/graph/lca_binary_lifting.html)
- Sryheni, S. (s/f). Lowest Common Ancestor of Two Nodes in a Tree. Baeldung.com. Recuperado el 27 de octubre de 2025, de <https://www.baeldung.com/cs/tree-lowest-common-ancestor>